

gRPC & Microservices Cheatsheet

Lessons: [27](#) — [gRPC & Microservices](#)

Examples: [27](#)

Covers: protobuf, code generation, the four RPC kinds, interceptors, status codes, deadlines

Legend: [*] = real API that the lesson has not covered yet

PROTOBUF: the contract

```
syntax = "proto3";
package orders.v1;                                versioned package
option go_package = "example.com/gen/orders/v1;ordersv1";

message Order {
    string id = 1;                                the FIELD NUMBER is the wire identity
    int64 amount_cents = 2;                       snake_case here, CamelCase in Go
    repeated Item items = 3;                       a slice
    map<string, string> labels = 4;                 a map
    optional string note = 5;                       presence: nil vs empty in Go
    reserved 6, 7;                                never reuse a removed field number
}

service OrderService {
    rpc GetOrder(GetOrderRequest) returns (Order);
    rpc ListOrders(ListOrdersRequest) returns (stream Order);
    rpc UploadEvents(stream Event) returns (UploadSummary);
    rpc Chat(stream Msg) returns (stream Msg);
}
```

CODE GENERATION

```
protoc --go_out=. --go-grpc_out=. orders.proto    the raw command
buf generate [*] the modern driver; buf.yaml + buf.gen.yaml
buf lint / buf breaking [*] style checks and backward-compatibility checks
google.golang.org/protobuf/cmd/protoc-gen-go      message code
google.golang.org/grpc/cmd/protoc-gen-go-grpc     service code
*.pb.go / *_grpc.pb.go                          generated – never edit, always commit
(the .proto file is the source of truth for BOTH sides of the wire)
```

SERVER

```
lis, err := net.Listen("tcp", ":50051")
s := grpc.NewServer(opts...)
pb.RegisterOrderServiceServer(s, &server{})
reflection.Register(s)    [*] lets grpcurl introspect the service
s.Serve(lis)              blocks
s.GracefulStop()          drain in-flight RPCs, then stop
s.Stop()                  [*] immediate

type server struct {
    pb.UnimplementedOrderServiceServer    EMBED this – forward compatibility
}

func (s *server) GetOrder(ctx context.Context, in *pb.GetOrderRequest)
    (*pb.Order, error) { ... }
```

CLIENT

```
conn, err := grpc.NewClient("dns:///orders:50051",
    grpc.WithTransportCredentials(insecure.NewCredentials()))
defer conn.Close()          ONE connection per service, reused everywhere
client := pb.NewOrderServiceClient(conn)
ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
defer cancel()
order, err := client.GetOrder(ctx, &pb.GetOrderRequest{Id: id})
grpc.WithDefaultCallOptions(...)  [*] message size limits, compression
grpc.WithChainUnaryInterceptor(...)  [*] client-side middleware
(the connection multiplexes over HTTP/2 – do NOT pool ClientConns yourself)
```

THE FOUR RPC KINDS

unary	one request -> one response (90% of real use)
server streaming	one request -> stream of responses
for { msg, err := stream.Recv(); if err == io.EOF { break } }	
client streaming	stream of requests -> one response
stream.Send(msg) ... stream.CloseAndRecv()	
bidirectional streaming	both directions, independently
stream.Send / stream.Recv from separate goroutines	
(streams are per-RPC; io.EOF from Recv means the peer finished cleanly)	

STATUS CODES

codes.OK	success
codes.InvalidArgument	the client sent something malformed (~400)
codes.NotFound	no such resource (~404)
codes.AlreadyExists	duplicate (~409)
codes.PermissionDenied	authenticated but not allowed (~403)
codes.Unauthenticated	no/invalid credentials (~401)
codes.ResourceExhausted	rate limited or quota (~429)
codes.FailedPrecondition	the system state forbids it
codes.Aborted	concurrency conflict; retry may work
codes.Unavailable	the service is down – RETRYABLE
codes.DeadlineExceeded	the caller's deadline passed
codes.Internal	your bug
codes.Unimplemented	the method doesn't exist here
status.Error(codes.NotFound, "order not found")	return this, not a bare error
status.FromError(err)	on the client: get the code back
st.Code() / st.Message()	
status.New(c, m).WithDetails(...)	[*] structured error details

METADATA, DEADLINES & CONTEXT

```
md := metadata.Pairs("authorization", "Bearer "+tok)
ctx = metadata.NewOutgoingContext(ctx, md)          client -> server
md, ok := metadata.FromIncomingContext(ctx)          server side
grpc.SendHeader / SetTrailer  [*] server -> client metadata
the DEADLINE propagates      a client timeout becomes the server's ctx deadline
always set a deadline        an RPC without one can hang forever
pass ctx down                to the DB call, the next RPC, everything
peer.FromContext(ctx)        [*] the caller's address
```

INTERCEPTORS (middleware)

```
func UnaryServerInterceptor(ctx, req, info, handler) (any, error) {
    start := time.Now()
    resp, err := handler(ctx, req)      call the actual method
    slog.Info("rpc", "method", info.FullMethod, "code", status.Code(err),
        "dur", time.Since(start))
    return resp, err
}

grpc.ChainUnaryInterceptor(logging, recovery, auth)    server side
grpc.StreamInterceptor(...) the streaming equivalent
what goes in one      logging, request-id, auth, metrics, recover, tracing
otelgrpc               [*] OpenTelemetry instrumentation, both sides
(recovery MUST be outermost – a panic in a handler kills the whole server)
```

TESTING & THE WIRE

```
bufconn                an in-memory listener: a real gRPC client and server
                        in ONE process, no ports, no flakes

lis := bufconn.Listen(1024*1024)
go s.Serve(lis)
grpc.NewClient("passthrough:///bufnet",
    grpc.WithContextDialer(func(ctx, _ string) (net.Conn, error) {
        return lis.DialContext(ctx) }), ...)

proto.Marshal(m)        the binary encoding – compact, and NOT self-describing
proto.Unmarshal(b, m)

protojson.Marshal(m)    [*] the canonical JSON form, for debugging and gateways
size comparison         protobuf is typically 3–10x smaller than the JSON
                        – the field NUMBER is on the wire, never the name

wrapping a ServerStream a stream interceptor cannot replace ctx directly;
                        embed grpc.ServerStream in your own type and override
                        Context() (and RecvMsg/SendMsg to observe messages)
```

MICROSERVICE CONCERNS

```
request id              generate at the edge, propagate in metadata
health checking         grpc_health_v1; Kubernetes can probe it directly
load balancing          client-side, via a resolver (dns:///) + round_robin
retries                 [*] service config with a retryPolicy; ONLY idempotent RPCs
circuit breaking        see the resilience sheet
mTLS                    [*] credentials.NewTLS for service-to-service identity
schema evolution        add fields; never renumber, never reuse, never change types
gRPC vs REST            gRPC inside the mesh, REST/JSON at the public edge
grpcurl -plaintext ... [*] curl for gRPC; needs reflection enabled
```

TRAPS & MEMORIZE

changing a field number	silently corrupts data – reserved exists for a reason
reusing a removed number	old clients decode garbage into the new field
not embedding Unimplemented	adding an RPC breaks compilation of every server
a ClientConn per call	defeats HTTP/2 multiplexing and leaks sockets
no deadline on an RPC	one slow dependency stalls every caller
returning a plain error	the client sees codes.Unknown with no detail
retrying non-idempotent RPCs	duplicate orders, duplicate charges
huge messages	the default limit is 4MB; stream instead of raising it
blocking in a stream handler	Send and Recv from one goroutine deadlocks a bidi stream
proto3 default values	0/"/false are indistinguishable from unset (use optional)
forgetting reflection	nothing can introspect the service in staging